

Reinforcement Learning for Decoding Repetition Codes

Aman Dutta
amandutta@princeton.edu

May 8, 2026

Abstract

Quantum error correction is very important for reliable quantum computation. However, decoding still remains extremely difficult even for simple codes. In this project, we formulate decoding for 3-qubit and 9-qubit repetition codes as an episodic Markov decision process and study whether reinforcement learning (RL) can learn decoding behavior. A tabular Q-learning agent is trained on the 3-qubit repetition code and then extended to the 9-qubit case. On the 3-qubit code, Q-learning matches a structured lookup decoder almost perfectly across a range of physical error rates. However, when scaling to the 9-qubit code, performance degrades significantly and becomes more variable, highlighting the limitations of naive tabular reinforcement learning. These results suggest that reinforcement learning provides a useful proof of concept for very small decoding problems, while also clearly exposing the challenges of scaling to larger quantum systems.

1 Introduction

Quantum information is very sensitive to noise. To prevent this and have working quantum computers, we need quantum error correction. Even if the logical state is prepared correctly, interactions with the environment or imperfect operations can introduce physical errors on the qubits. Quantum error-correcting codes address this by encoding logical information across many physical qubits, so that errors can be detected and corrected without directly measuring the encoded logical state [?, ?]. Decoding is the step where we use measured syndrome information to decide what correction should be applied. A good decoder is therefore essential: even if the code can in principle protect information, the recovery only works if the decoder chooses the right correction often enough.

One way to think about decoding is as an inference problem. The decoder sees a syndrome and tries to infer the most likely underlying error. However, we can also view it as a sequential decision-making problem. The decoder observes the current syndrome, applies a correction action, and the system may continue to evolve under noise. **The final goal is not just to reduce the syndrome locally, but to end the episode with the encoded state successfully restored. This viewpoint makes reinforcement learning a natural framework to explore, since reinforcement learning studies how an agent can learn a policy by interacting with an environment and receiving rewards [?, ?].**

In this project, I use repetition codes to study this. Repetition codes protect against bit-flip errors and have a very interpretable syndrome structure, which makes them useful for a first decoding experiment. The hidden environment state is a binary error pattern over the physical qubits, while the observed state is the syndrome from adjacent parity checks. Since the correct decoding rule for small repetition codes is easy to understand, the learned reinforcement learning

policy can be compared directly against a structured minimum-weight lookup decoder. This gives a clear baseline and keeps the project focused on whether the learning setup itself is working.

I start with the 3-Qubit Repetition code, which was the original scope of this project because of its small state and action space. There are only four possible syndromes, so a tabular Q-learning agent can explore the environment thoroughly and should be able to learn how to apply correction. This project also applies RL to 9-Qubit Repetition code, which has much larger action space compared to 3-qubit one. Moving from three to nine qubits increases the syndrome space from 4 states to $2^8 = 256$ states, which is still small compared with realistic codes but already much harder for a tabular method. Since tabular Q-learning stores separate values for each state-action pair, it does not automatically generalize between similar syndromes.

The main question of this project is therefore modest: can tabular Q-learning learn decoding behavior for small repetition codes, and where does this simple approach start to break down? The results show that tabular Q-learning works very well on the 3-qubit repetition code and nearly matches the minimum-weight lookup decoder across the tested error rates. However, when the same framework is extended to the 9-qubit repetition code, performance drops substantially and becomes less stable.

2 Problem Formulation

2.1 Repetition-Code Setting

We consider decoding for repetition codes that protect a single logical bit against bit-flip errors by encoding it across an odd number of physical qubits. Let n denote the code length. The underlying physical state is a binary error pattern

$$e \in \{0, 1\}^n, \quad (1)$$

where $e_i = 1$ indicates that qubit i has experienced a bit-flip error. This error pattern is not directly observable.

The observable state is the syndrome defined by adjacent parity checks:

$$s_i = e_i \oplus e_{i+1}, \quad i = 0, \dots, n-2, \quad (2)$$

so that $s \in \{0, 1\}^{n-1}$. Each nonzero entry of s corresponds to a domain wall in the error configuration. The mapping $e \mapsto s$ is many-to-one, so the decoder must infer a correction from incomplete information.

This work focuses on the cases $n = 3$ and $n = 9$. For $n = 3$, the syndrome has length 2, giving $2^2 = 4$ possible syndromes. This small state space makes it possible to enumerate all configurations and directly interpret the decoding rule. For $n = 9$, the syndrome has length 8, giving $2^8 = 256$ possible syndromes. This small increase already makes the learning problem substantially harder.

2.2 MDP Formulation

We formulate decoding as an episodic Markov decision process in which the agent interacts with the environment through syndrome observations and correction actions. At each step, the agent observes the current syndrome $s \in \{0, 1\}^{n-1}$ and selects an action

$$a \in \{0, \text{flip } i \mid i = 0, \dots, n-1\}, \quad (3)$$

where $a = 0$ denotes no operation and $a = \text{flip } i$ applies a bit-flip to qubit i .

The environment evolves according to

$$e' = e \oplus a \oplus \xi, \quad \xi_i \sim \text{Bernoulli}(p), \quad (4)$$

where a is interpreted as a binary vector indicating which qubit is flipped, and ξ represents independent bit-flip noise applied after each action. The next observation is obtained by computing the syndrome of the updated error pattern.

The reward is defined at the end of the episode:

$$r = \begin{cases} 1 & \text{if } e_T = 0, \\ 0 & \text{otherwise.} \end{cases} \quad (5)$$

This sparse reward structure emphasizes whether the decoder successfully restores the encoded state, rather than intermediate progress.

Episodes are run for a fixed horizon $T = n$, so the number of decision steps scales with the code length. This provides a consistent setting across different system sizes while allowing multiple opportunities for correction.

The noise model is restricted to independent bit-flip errors, and syndrome observations are assumed to be perfect. The dynamics are defined directly on the error pattern rather than through circuit-level operations. These assumptions keep the environment interpretable and aligned with the repetition-code setting, while isolating the learning problem from additional experimental complexity. [?, ?].

3 Methods

3.1 Environment and Noise Model

The decoding environment evolves in discrete time over a fixed-length episode. At each step, the agent observes the syndrome and selects a correction action, which is applied to the underlying error pattern. The system then evolves under additional noise before the next observation.

The noise model is independent bit-flip noise. After each action, each qubit flips with probability p , so that

$$\xi_i \sim \text{Bernoulli}(p), \quad e' = e \oplus a \oplus \xi. \quad (6)$$

This choice matches with the repetition-code setting, which is designed to detect and correct bit-flip errors [?]. Using a more general noise model would introduce errors that the code is not intended to handle.

Episodes are run for $T = n$ steps. The final error configuration determines whether decoding succeeds.

3.2 Lookup Decoder Baseline

As a baseline, we use a deterministic lookup decoder that selects a minimum-weight error pattern consistent with the observed syndrome. Given a syndrome s , the decoder constructs an error configuration $\hat{e}$ satisfying

$$s_i = \hat{e}_i \oplus \hat{e}_{i+1}, \quad (7)$$

and applies a correction based on $\hat{e}$.

For $n = 3$, this reduces to a simple mapping between syndromes and single-qubit flips. For larger repetition codes, the same rule extends by selecting a minimum-weight consistent configuration.

3.3 Tabular Q-Learning

We use tabular Q-learning to learn a decoding policy [?, ?]. The action-value function is updated according to

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right], \quad (8)$$

with $\alpha = 0.1$ and $\gamma = 0.95$.

Actions are selected using an ϵ -greedy policy:

$$a = \begin{cases} \text{random} & \text{with prob. } \epsilon, \\ \arg \max_a Q(s, a) & \text{otherwise,} \end{cases} \quad (9)$$

with $\epsilon = 0.1$. We also consider a UCB-style exploration strategy with constant $c = 1.25$.

To provide additional training signal, we optionally include potential-based reward shaping based on the syndrome weight [?]. The main objective remains the terminal reward.

3.4 Scaling from 3 to 9 Qubits

For $n = 3$, the state-action space contains $4 \times 4 = 16$ entries. For $n = 9$, it increases to $256 \times 10 = 2560$. Since tabular Q-learning does not generalize across states, each entry must be learned independently. This makes exploration significantly less efficient in the larger system.

4 Experiments

The experiments compare a structured lookup decoder with a tabular Q-learning decoder on repetition codes of length $n = 3$ and $n = 9$. The 3-qubit code is small enough that the syndrome-action space can be explored thoroughly, while the 9-qubit code provides a larger but still manageable test of how the same learning setup behaves as the problem size increases.

The main evaluation uses physical bit-flip rates

$$p \in \{0.01, 0.03, 0.05, 0.08, 0.10\}.$$

For each value of p , the lookup decoder is evaluated directly and the Q-learning agent is trained from scratch and then evaluated on the same environment. The training budget is 5000 episodes for the 3-qubit code and 10000 episodes for the 9-qubit code. Reported success rates are averaged over 3 random seeds, with 2000 evaluation episodes per seed.

The primary experiment is a success-rate sweep across error rates. This experiment measures how closely the learned decoder matches the structured baseline as the physical noise level increases. Because both decoders are tested under the same transition dynamics, the comparison isolates the effect of learning rather than differences in the environment.

In addition to final success rates, I record training curves for the Q-learning agent. These curves track performance over the course of training and are used to compare learning speed and stability for the 3-qubit and 9-qubit codes. This makes it possible to distinguish poor final performance from slow or unstable convergence.

I also include a strategy comparison at a fixed error rate of $p = 0.05$. In this experiment, the Q-learning agent is run with three variants: standard ϵ -greedy exploration, ϵ -greedy with potential-based reward shaping, and UCB exploration. Reward shaping uses syndrome weight as a simple potential function, while UCB replaces random exploration with an optimism-based action-selection rule. These variants are included as small ablations rather than as a full hyperparameter search.

Together, these experiments evaluate three aspects of the decoder-learning problem: final decoding performance across noise levels, training behavior over time, and sensitivity to exploration and reward design. This setup is sufficient to test both whether tabular Q-learning works in the small 3-qubit setting and how sharply its performance changes in the larger 9-qubit case.

5 Results

5.1 Main Success-Rate Sweep

For the 3-qubit code, Q-learning matches the lookup decoder across all error rates. The difference remains below 0.003 throughout the sweep. For example,

$$p = 0.05 : \quad 0.8505 \text{ (lookup)}, \quad 0.8480 \text{ (Q-learning)}. \quad (10)$$

Similar agreement holds at all values of p .

For the 9-qubit code, Q-learning performs substantially worse. At $p = 0.01$,

$$0.9098 \text{ (lookup)} \quad \text{vs} \quad 0.7147 \text{ (Q-learning)}, \quad (11)$$

and at $p = 0.05$,

$$0.5607 \quad \text{vs} \quad 0.4523. \quad (12)$$

The gap increases as the noise level grows.

Figure ?? shows these trends. For $n = 3$, the curves nearly overlap. For $n = 9$, Q-learning remains consistently below the lookup decoder.

5.2 Training Dynamics

The training curves reflect the difference in problem size. For $n = 3$, performance converges quickly to a stable value. For $n = 9$, convergence is slower and more variable, with a lower final plateau.

Figure ?? shows that the larger state space leads to noisier updates and less stable learning, consistent with sparse rewards and limited state coverage.

5.3 Strategy Comparison

At $p = 0.05$, the exploration strategies perform similarly for $n = 3$:

$$0.848 \text{ (ϵ -greedy)}, \quad 0.820 \text{ (shaping)}, \quad 0.848 \text{ (UCB)}. \quad (13)$$

For $n = 9$, the differences are larger:

$$0.452 \text{ (ϵ -greedy)}, \quad 0.440 \text{ (shaping)}, \quad 0.082 \text{ (UCB)}. \quad (14)$$

Figure ?? shows that UCB performs particularly poorly in the larger state space. Reward shaping does not improve the mean success rate in either case.

6 Discussion and Limitations

The results show a clear difference between the small and larger repetition-code settings. In the 3-qubit case, tabular Q-learning performs almost identically to the structured lookup decoder across all error rates. This is not too surprising given how small the problem is. The syndrome space has only four states, and the action space is equally small, so the agent can effectively explore the entire state-action space during training. In this regime, Q-learning is able to estimate values reliably and ends up learning a policy that is very close to the minimum-weight decoding rule. In that sense, the 3-qubit experiment mainly serves as a sanity check that the MDP formulation and learning setup are working as intended [?, ?].

The 9-qubit case is qualitatively different. Increasing the code size from three to nine qubits expands the syndrome space from 4 states to 256 states, and also increases the number of available actions. Even though this is still a relatively small problem compared to realistic quantum error-correction settings, it is already large enough that tabular Q-learning struggles. The agent no longer visits all states frequently, and the value estimates become noisy. Combined with the sparse terminal reward, this makes it much harder for the agent to learn a reliable decoding policy.

One useful way to interpret this is in terms of data efficiency. The lookup decoder directly uses the structure of the repetition code. Given a syndrome, it constructs a minimum-weight error configuration and applies a correction based on that. This essentially encodes prior knowledge about how the code works [?]. The Q-learning agent, on the other hand, has to infer this mapping purely from interaction with the environment. As the number of states increases, the amount of data needed to learn this mapping grows quickly, and the tabular approach does not provide any mechanism for generalizing across similar syndromes. This is why the gap between Q-learning and the lookup decoder becomes much more visible in the 9-qubit setting.

The exploration and shaping experiments point in the same direction. In the 3-qubit case, different exploration strategies all perform similarly, which is expected since the agent can already explore the entire space. In the 9-qubit case, the differences are more noticeable, but none of the simple modifications fully resolve the problem. Reward shaping based on syndrome weight does not meaningfully improve the mean success rate, suggesting that this heuristic does not provide enough useful information beyond the terminal reward. The UCB-based exploration strategy performs especially poorly, indicating that encouraging additional exploration does not necessarily help when the agent cannot effectively use the additional data it collects. This behavior is consistent with known limitations of tabular methods in larger state spaces [?].

Even though the reinforcement learning approach does not outperform the structured baseline, the results are still useful. The 3-qubit experiment shows that the learning formulation is capable of recovering a known decoding rule in a simple setting. The 9-qubit experiment then shows where this approach starts to break down. Taken together, these results make the scaling issue explicit: reinforcement learning provides a flexible way to formulate decoding as a sequential decision problem, but naive tabular methods do not scale well without additional structure or more expressive representations. In that sense, the contribution of this project is not a new decoder, but a controlled demonstration of the limits of a simple learning-based approach.

6.1 Limitations

There are several important limitations to this study. First, the noise model is restricted to independent bit-flip errors. This is appropriate for repetition codes, but it is much simpler than more realistic noise models that include phase errors or correlated noise. As a result, the conclusions here should not be interpreted as applying directly to more general quantum error-correction

settings [?, ?].

Second, the agent is assumed to observe perfect syndrome measurements. In practice, syndrome extraction can itself be noisy, and decoding often requires combining information from multiple rounds of measurements. This is not captured in the current formulation, which treats the syndrome as a fully reliable observation.

Third, the environment abstracts away circuit-level details and fault-tolerant constraints. The decoding process is modeled as a sequence of abstract actions applied to an error pattern, rather than as a sequence of physical operations subject to hardware limitations. Similarly, although the 9-qubit experiment involves nine qubits, it is still a repetition-code study and should not be interpreted as a simulation of the 9-qubit Shor code or a full fault-tolerant decoding procedure.

Finally, the use of tabular Q-learning itself imposes a strong limitation. The Q-table grows with the number of syndrome-action pairs, and the method does not generalize across states. This leads to poor scalability as the code size increases. The results in the 9-qubit case already reflect this issue, even in a relatively small setting. For larger or more realistic problems, more advanced methods that can exploit structure or generalize across similar states would likely be necessary.

Overall, the conclusions of this work should be understood within the narrow setting of repetition-code decoding with a simple tabular reinforcement learning approach. The results highlight both what this approach can do in small systems and where it begins to fail as the problem becomes larger.

References